

Design and Implementation of a solution for the uniform word problem for ortholattices

ALESSIO GERARDO DEMATTIA

1 INTRODUCTION

The word problem for groups is a central problem in both group theory and computability theory. It focuses on determining whether two terms in a given group represent the same element based on the group's axioms and a finite set of generators. Specifically, the problem involves finding an algorithm that, given two terms in the group's signature, decides if they are equal under the group's operations and relations.

A more general variant of this problem is the **uniform word problem**. In this case, the input includes not only the two terms to compare but also a finite presentation P . The presentation P consists of a set of equality and inequality relations among terms in the group's signature. The task is to find an algorithm that decides whether the two given terms are equal under the axioms defined by P .

It is well-known that, in general, the word problem is undecidable for arbitrary groups. However, there exist specific algebraic classes for which the word problem is decidable. One such class is **ortholattices**.

In the context of ortholattices, the word problem with finite presentation becomes particularly interesting when the presentation consists of *inequalities* between terms. For example, given a presentation P consisting of inequalities of the form $a \leq b$, the task is to determine whether a derived inequality $x \leq y$ holds under the axioms of ortholattices combined with the constraints provided by P .

In this dissertation, we will begin by recalling the fundamental definitions of ortholattices, followed by presenting the key theoretical results that will facilitate the creation of the algorithm. We will discuss the specific implementations, present tests, and propose measures to enhance efficiency.

2 THEORETICAL BACKGROUND

Ortholattices form an intriguing class of algebras. By weakening the distributivity in Boolean algebras, ortholattices reveal fascinating applications across various fields, ranging from quantum mechanics to automated reasoning. Ortholattices are also closely related to lattices that are derived from additional assumptions. The knowledge of an algorithm for solving the word problem in lattices is well-established; this is the renowned Whitman Algorithm [3], which efficiently solves the problem in polynomial time relative to the length of the terms.

2.1 Ortholattices - Axiomatization

In order theory, the term *ortholattice* refers to a bounded lattice (a lattice with a least element 0 and a greatest element 1) that satisfies the orthocomplementation property: every element of the lattice has a complement (a negated counterpart).

Formally the classical axiomatization of ortholattices in the signature $(S, \leq, \wedge, \vee, \neg, 0, 1)$, where S is the generator set, can be stated as follows:

The axioms of partial order:

$$\begin{aligned}
x &\leq x \quad (\text{Reflexivity}) \\
x &\leq y \ \& \ x \leq y \iff x = y \quad (\text{Antisymmetry}) \\
x &\leq y \ \& \ y \leq z \implies x \leq z \quad (\text{Transitivity})
\end{aligned}$$

In the transitivity axiom, the middle term y is referred to as the cut term.
The axioms of bounded lattice:

$$\begin{aligned}
x \wedge y &\leq x \ \text{and} \ x \wedge y \leq y \quad (\text{L}\wedge) \\
x &\leq x \vee y \ \text{and} \ y \leq x \vee y \quad (\text{R}\vee) \\
z &\leq x \ \& \ z \leq y \implies z \leq x \wedge y \quad (\text{R}\wedge) \\
x &\leq z \ \& \ y \leq z \implies x \vee y \leq z \quad (\text{L}\vee) \\
0 &\leq x \quad (\text{Min}) \\
x &\leq 1 \quad (\text{Max})
\end{aligned}$$

Notation: $\wedge$ is also called the meet operator, and $\vee$ is called the join operator.
The axioms of orthocomplementation:

$$\begin{aligned}
x &\leq y \implies \neg y \leq \neg x \quad (\text{Rev}) \\
x &\leq \neg\neg x \ \text{and} \ \neg\neg x \leq x \quad (\text{R}\neg\neg, \text{L}\neg\neg) \\
x \wedge \neg x &\leq 0 \quad (\wedge - 0) \\
1 &\leq x \vee \neg x \quad (1 - \vee)
\end{aligned}$$

The axioms $(\wedge - 0)$ and $(1 - \vee)$ can be replaced by [1]:

$$x \leq \neg x \implies x \leq y \quad (\text{NC})$$

We will use the last axiom in the formulation of our Tactic. In the context of derivation rules, we define a 0-premises axiom as an axiom that does not involve any implications. These are basic axioms that stand independently and do not require any premises to be applied.

The ortholattices are less expressive than boolean algebra, but strong enough to prove important results like the De Morgan's rules:

$$\neg(x \wedge y) = \neg x \vee \neg y \ \text{and} \ \neg(x \vee y) = \neg x \wedge \neg y$$

2.2 Theoretical Decidability for ortholattices

The problem discussed is to determine whether the inequality $x \leq y$ holds under a finite set P of relations and the axioms of ortholattices. To discuss about the theoretical results underlying the algorithm, we have to introduce few definition:

Definition 2.1. The subterms of $\neg x$ are x and all subterms of x . The subterms of $x \vee y$ and $x \wedge y$ are x, y and all their subterms

Definition 2.2. We define a *known term* as any term that appears either in the axioms of the presentation or in the conclusion, along with all of its subterms.

Definition 2.3. We define a *complemented term* as any term of the form $\neg x$ or $\neg\neg x$, where x is a known term.

Definition 2.4. We define *relevant term* as any term that is either a known term or a complemented term.

The classical derivation in sequent calculus for an atomic formula of the form $x \leq y$ is nothing more than a tree where each node has at most two parents. Several interesting observations can be made about these derivations. For instance, any derivation in which the same sequent appears twice on the same branch can be reduced to a form where each formula appears only once per branch, a property known as loop-free derivation (in other words, we do not need a specific sequent to prove the same sequent). Formulas of the type $x \leq 1$, $x \leq x$, $0 \leq x$, $\neg\neg x \leq x$ and $x \leq \neg\neg x$ can be derived in a single step.

The main results of [1], which have a counterpart in the Partial Cut Theorem of [2], state that:

LEMMA 2.5. *Any given derivation can be transformed into one where the transitivity rule utilize as cut term only relevant term (known terms and complemented terms).*

LEMMA 2.6. *Any derivation containing a known term with more than a double negation can be reduced to an equivalent derivation where no term has more than a double negation.*

THEOREM 2.7 (THE UNIFORM WORD PROBLEM). *The derivability of an atom formula, i.e. $x \leq y$, from a finite number of given atoms in presentation P is decidable.*

Proving this is fairly straightforward: it suffices to explore all derivations that use only relevant terms and are loop-free on each branch. Since the set of relevant terms is finite, this results in a finite space of derivation trees, making the problem decidable.

3 ALGORITHM AND IMPLEMENTATION

Listing all possible derivation trees and checking each one to decide whether an atomic formula is derivable is a functional strategy but highly computationally expensive. The main idea consists of using a backward algorithm inspired by the orthologic algorithm presented in [2]. The algorithm has been designed to be compatible with the LISA proof assistant, leveraging its structures, which will be referenced throughout this discussion.

Algorithm 1: Backward Algorithm for the Uniform Word Problem

Input: A sequent $P \vdash x \leq y$

```

1 Initialize variables;
2 relevant_term  $\leftarrow$  extractRelevantTerm( $P \vdash x \leq y$ );    // Set of relevant terms
3 visited_sequent  $\leftarrow \emptyset$ ;                                // Set of already visited sequents
4 proven_sequent  $\leftarrow \{P \vdash a \leq b \mid (a \leq b) \in P\}$ ;    // Set of proven sequents

5 Define prove(goal: Sequent);
6 begin
7   if goal  $\in$  proven_sequent then
8     | return ValidProofTactic;
9   end
10  else if goal  $\in$  visited_sequent then
11    | return InvalidProofTactic;
12  end
13  else
14    visited_sequent  $\leftarrow$  visited_sequent  $\cup \{\text{goal}\}$ ;
15    //  $\emptyset$ -premises checks
16    reflexivity(goal);
17    double_negation(goal);
18    least_element(goal);
19    greatest_element(goal);
20    // Backward checks
21    negation(goal);
22    left_meet(goal);
23    left_join(goal);
24    right_meet(goal);
25    right_join(goal);
26    NC(goal);
27    // Transitivity check
28    current_relevant_term  $\leftarrow$  extractRelevantTerm(goal);
29    useful_relevant_term
30     $\leftarrow$  relevant_term.intersect(current_relevant_term);
31    transitivity(goal, useful_relevant_term);
32    return InvalidProofTactic;
33  end
34 end
35 prove( $P \vdash x \leq y$ );

```

The algorithm performs a backward search. At the beginning, several variables are initialized. The **relevant_term** variable contains all relevant terms, which are efficiently extracted using the `extractRelevantTerm` function. The **visited_sequent** variable stores all sequents that have been visited but not yet proven, ensuring loop-free derivations by avoiding redundant exploration. The

proven_sequent variable tracks all sequents that have already been proven. Both **visited_sequent** and **proven_sequent** are dynamically updated throughout the recursive process. Regarding the practical implementation, **proven_sequent** is a set of **ValidProofTactic** objects that can be called to obtain additional **ProofTacticJudgment**, while **visited_sequent** is a set of sequents that have already been visited.

Within the code, there are two types of blocks:

- **0-premises blocks:** These directly check if the conclusion matches a specific form. Examples include: .
- **Recursive call blocks:** These involve recursive calls to explore further derivations. Examples include: .

For efficiency, to minimize the length of the proof, the 0-premises blocks are placed at the beginning of the process, while the more computationally expensive **Transitivity block** is deferred to the end.

Below we present an example of block (Transitivity block); the others one are similar.

Algorithm 2: Transitivity Block

Input: A sequent $goal: P \vdash x \leq y$ and **useful_relevant_term**

Output: A proof tactic if the sequent is provable.

```

1  foreach elem ∈ useful_relevant_term do
2      if elem ≠ left and elem ≠ right then
3          goal_1 ← (P ⊢ elem ≤ y);
4          proofTactic_1 ← prove(goal_2);
5          if proofTactic_1 is valid then
6              goal_1 ← P ⊢ x ≤ elem;
7              proofTactic_2 ← prove(goal_2);
8              if proofTactic_1 is valid then
9                  proofTactic ←;
10                 proven_sequent ← proven_sequent ∪ {proofTactic};
11                 return the proof of goal;
12             end
13         end
14     end
15 end

```

The algorithm presented above 1 decides any inequality atomic formula under a finite presentation; it is generalized to solve equality atomic formula too. Currently, there is a restriction on the type of presentation; only inequalities can be given. It is not difficult to generalize this to any type of presentation.

3.1 Testing

The code has been tested on various problems, initially on Whitman solvable problems within the world of non-complemented lattices. Then on problems related to ortholattices with no axioms in presentation, this class of problems is already efficiently solvable with normalization algorithms.

- (1) **semiDistributivity:** $(x \vee (y \wedge z)) \leq ((x \vee y) \wedge (x \vee z))$
- (2) **meetAssociative:** $(x \wedge (y \wedge z)) = ((x \wedge y) \wedge z)$
- (3) **idempotence:** $(x \vee x) = x$
- (4) **ExFalsoSequiturQuodlibet:** $x \wedge \neg x \leq 0$

- (5) **TertiumNonDatur**: $1 \leq x \vee \neg x$
- (6) **DeMorgan1**: $(\neg x \wedge \neg y) = \neg(x \vee y)$
- (7) **DeMorgan2**: $\neg(x \wedge y) = (\neg x \vee \neg y)$
- (8) **normalization**: $((x \wedge \neg(w \wedge \neg w)) \vee (y \wedge (\neg z \vee z))) = (y \vee (x \wedge 1))$
- (9) **transitivityTrain**: $(x \leq y, y \leq z, z \leq w, w \leq t) \vdash x \leq t$
- (10) **orthologicExample**: $1 \leq (x \wedge (\neg x \vee y)) \vdash 1 \leq y$
- (11) **doubleBound**: $(x \leq \neg z, y \leq \neg z) \vdash ((x \vee y) \wedge z) = 0$
- (12) **logicApplication**: $1 \leq ((\neg x \vee (\neg y \wedge z)) \wedge (\neg(\neg y \wedge z) \vee x)), ((\neg x \vee (\neg y \wedge z)) \wedge (\neg(\neg y \wedge z) \vee x)) \leq 1, (x \wedge (\neg(\neg w) \vee \neg z)) \leq 1, 1 \leq (x \wedge (\neg(\neg w) \vee \neg z)) \vdash (\neg y \wedge w) = 1$

All the tests (and more) are available in the single project file present in this GitHub repository. The file also includes the axioms and the algorithm as a ProofTactic.

4 EFFICIENCY AND FURTHER RELATED WORK

The code is functional, although there is room for improvement in terms of efficiency. Several strategies have been implemented to optimize specific parts. For instance, every time the algorithm executes the transitivity block, the possible cut terms are calculated as the relevant terms of the formula currently being proved. Additionally, a flag can be introduced to prevent another use of transitivity or a left operation [1] immediately after a backward transitivity step. A potential future step is to utilize normalization in the context of ortholattices to reduce the size (and consequently the number of relevant terms) of the axioms. By minimizing the length of the terms, it becomes possible to decrease the number of operations to be performed, particularly in the transitivity block. In conclusion, this code solves the uniform word problem in the context of ortholattices and opens up significant potential applications, including its use in various domains such as the study of formula satisfiability. Ortholattices provide a good approximation of Boolean algebra, making this approach highly relevant.

REFERENCES

- [1] Andrea Meinander. 2010. A solution of the uniform word problem for ortholattices. *Mathematical Structures in Computer Science* 20 (2010), 625–638.
- [2] Viktor Kuncak Simon Guilloud. 2024. Orthologic with Axioms. *Proceedings Of The Acm On Programming Languages-Pacmpl* 8 (2024).
- [3] Philip M. Whitman. 1941. Free Lattices. *Annals of Mathematics* 42 (1941), 325–330.